

Aide-Mémoire Python du Cours

Ce document regroupe les concepts de programmation, les fonctions intégrées et les bibliothèques que nous avons abordés dans ce cours.

1. Variables, Types et Structures de Données

Les variables stockent des informations. Le type de la donnée modifie les opérations que l'on peut effectuer.

- **int** (entier) : Nombres entiers (ex. 10, -3).
- **float** (flottant) : Nombres à virgule (ex. 3.14, -0.5).
- **str** (chaîne de caractères) : Texte placé entre guillemets ou apostrophes (ex. "Bonjour", 'A').
 - **f-strings** (chaînes formatées) : `f"Valeur : {x}"` évalue la variable `x` directement dans la chaîne.
 - Méthodes utiles : `.startswith(prefix)`, `.split(sep)` (découpe une chaîne en liste).
- **list** (liste) : Collection ordonnée et modifiable d'éléments (ex. [1, 2, 3]).
 - `.append(element)` : Ajoute un élément à la fin de la liste.
- **tuple** (tuple) : Collection ordonnée mais **non modifiable** d'éléments (ex. (1, 2)).

Slicing (Découpage) : Permet d'extraire une partie d'une chaîne, d'une liste ou d'un tableau NumPy.

- Syntaxe : `collection[début:fin:pas]` (la `fin` est exclue).
- Exemples : `liste[0:3]` (3 premiers éléments), `liste[::-1]` (inverser la liste).

Type et Conversion :

- `type(obj)` : Retourne le type de l'objet.
 - `int(x)`, `float(x)`, `str(x)`, `list(x)` : Convertissent `x` vers le type spécifié.
-

2. Opérateurs Mathématiques

- `+` (Addition), `-` (Soustraction)
- `*` (Multiplication), `/` (Division flottante)
- `//` (Division entière, ex. `5//2` donne 2)
- `**` (Puissance, ex. `2**3` donne 8)
- `%` (Modulo, reste de la division entière, ex. `5%2` donne 1)

(Note : Pour les opérations avancées sur des séries de données, nous utilisons systématiquement NumPy plutôt que le module `math` de base).

3. Structures de Contrôle

Les structures de contrôle gèrent le flux d'exécution du code : conditions et boucles.

Conditions

Elles permettent d'exécuter du code différemment selon qu'un test est Vrai ou Faux.

```
if condition:
    # exécuté si la condition est Vraie
elif autre_condition:
    # exécuté si la première est Fausse et celle-ci est Vraie
else:
    # exécuté si aucune des conditions n'est Vraie
```

Boucles

- **for** (boucle pour) : Parcourt une collection (liste, tableau) ou un nombre d'itérations connu.
 - `range(début, fin, pas)` : Génère une séquence d'entiers. Idéal avec `for` (ex. `range(5)` donne 0 à 4).
 - `enumerate(iterable)` : Parcourt une collection en renvoyant à la fois l'**indice** et la **valeur** (`for index, val in enumerate(liste):`).
- **while** (boucle tant que) : Répète un bloc de code tant qu'une condition reste Vraie.

4. Fonctions Personnalisées

Créer ses propres fonctions avec le mot-clé `def`.

```
def ma_fonction(arg1, arg2):  
    """  
    Docstring (Chaîne de documentation) :  
    Explique ce que fait la fonction, ses arguments et sa valeur de retour.  
    """  
    resultat = arg1 + arg2  
    return resultat
```

5. Fonctions Intégrées (Built-ins)

- `print(valeur, ...)` : Affiche les valeurs dans la console.
 - `len(obj)` : Renvoie le nombre d'éléments d'une liste, d'un tableau, d'une chaîne, etc.
 - `sum(obj)`, `min(obj)`, `max(obj)` : Somme, minimum et maximum d'une collection. (*En pratique, on privilégie `np.sum()`, `np.min()`, `np.max()` avec NumPy.*)
 - `abs(x)` : Renvoie la valeur absolue.
 - `round(nombre, ndigits)` : Arrondit nombre avec ndigits chiffres après la virgule.
 - `repr(obj)` : Renvoie une représentation textuelle (souvent détaillée) d'un objet.
 - `open(fichier, mode)` : Ouvre un fichier (ex. `mode="r"` pour lecture).
-

6. Bibliothèques Externes (Modules)

Les bibliothèques nécessitent un `import` avant d'être utilisées.

NumPy (`import numpy as np`)

Bibliothèque de référence pour la manipulation de tableaux (arrays) et le calcul numérique.

Création et Manipulation de Tableaux :

- `np.array(liste)` : Convertit une liste Python en tableau NumPy. L'argument `order='C'` ou `order='F'` contrôle si les valeurs sont stockés en ligne-majeur ou colonne-majeur.
- `np.arange(start, stop, step)` : Crée un tableau de nombres avec un pas constant.
- `np.linspace(start, stop, num)` : Crée un tableau de num valeurs réparties uniformément entre start et stop.
- `np.zeros(shape)` / `np.ones(shape)` : Crée un tableau rempli de zéros ou de uns avec les dimensions shape (ex. (3, 3)).
- `np.ones_like(a)` : Crée un tableau de uns avec la même forme (dimensions) que le tableau a.
- `np.meshgrid(x, y)` : Génère des grilles de coordonnées N-D depuis des vecteurs 1D (essentiel pour les graphiques 3D).
- `np.stack` : Empile des tableaux, comme des rangs avec `axis=0` ou comme des colonnes avec `axis=1`.
- `np.block` : Crée une matrice avec une assemblage des sous-matrices.
- `np.column_stack(tup)` / `np.concatenate(tup, axis)` : Assemble des tableaux (en colonnes, ou bout à bout).
- `np.where(condition, x, y)` : Retourne les éléments choisis dans x ou y (ou leurs indices) en fonction de la condition.
- `np.transpose(A)` : Tranpose un vecteur ou une matrice A (change les rangs pour colonnes et les colonnes pour rangs).
- `A.T` : Équivalent à `np.transpose(A)`.
- `A.strides` : Retourne le nombre d'octets à se déplacer pour accéder à la prochaine ligne et prochaine colonne.

Algèbre Linéaire et Matrices :

- `@` : Fait une multiplication matricielle.
- `np.eye(n)` : Crée une matrice d'identité de taille n.
- `np.diag(v)` : Extrait la diagonale d'un tableau ou construit une matrice diagonale.
- `np.trace(a)` : Calcule la somme de la diagonale d'une matrice.

- **np.linalg.norm(x)** : Calcule la norme (longueur) d'un vecteur ou d'une matrice. L'argument **order=p** donne la norme L^p , de 1 jusqu'à l'infinie (avec **np.inf**)
- **np.linalg.solve(A, b)** : Résout de manière exacte le système d'équations linéaires matricielles $Ax = b$.
- **np.linalg.lstsq(a, b)** : Donne la solution des moindres carrés pour $ax = b$ (régression linéaire). Retourne le vecteur solution **x**, les résidus, le rang de **a** et les valeurs singulières de **a**.
- **np.linalg.cond(x)** : Calcule le conditionnement d'une matrice.
- **np.linalg.eig(A)** : Calcule les valeurs propres et les vecteurs propres d'une matrice carrée **A**.
- **np.linalg.eigh((A, B))** : Calcule les valeurs propres et les vecteurs propres d'un problème des valeurs propres généralisée de la matrice **A** par rapport de la matrice **B**.

Statistiques, Maths et Ajustements :

- **np.mean(a)**, **np.std(a)**, **np.cov(m)** : Calculent respectivement la moyenne, l'écart-type et la matrice de covariance.
- **np.sum(a)**, **np.min(a)**, **np.max(a)**, **np.argmax(a)** : Opérations globales (somme, min, max, indice du max).
- **np.exp(x)**, **np.log(x)**, **np.log10(x)**, **np.sqrt(x)**, **np.round(a)** : Fonctions mathématiques sur tableaux complets.
- **np.inf**, **np.pi** : Valeurs spéciales.
- **np.polyfit(x, y, deg)** : Régression polynomiale ; retourne les coefficients (ex. **deg=1** pour une droite).
- **np.polyval(p, x)** : Évalue le polynôme de coefficients **p** (tel que retourné par **np.polyfit**) aux points **x**.
- **np.interp(x, xp, fp)** : Interpolation linéaire 1D. Renvoie les valeurs interpolées aux positions **x** depuis les points de contrôle (**xp**, **fp**). Ne peut pas extrapoler (valeurs clampées aux extrémités).

Précision, Différences et Mémoire :

- **np.allclose(a, b, rtol, atol)** : Vérifie si les éléments de deux tableaux sont "égaux" (très proches), palliant les imprécisions des flottants.
- **np.shares_memory(a, b)** : Vérifie si deux tableaux pointent vers les mêmes données en RAM.
- **np.float32 / np.finfo(dtype)** : Type de nombre flottant simple précision, et fonction pour inspecter ses limites numériques (epsilon, min, max).

Aléatoire (np.random) :

- **np.random.default_rng(seed)** : Crée un générateur moderne (**Generator**) recommandé pour la reproductibilité et l'isolation des tirages aléatoires.
 - **rng.uniform(a, b, size)** : Tirages uniformes entre **a** et **b**.
 - **rng.normal(loc, scale, size)** : Tirages gaussiens.
 - **rng.exponential(scale, size)** : Tirages exponentiels.
- **np.random.seed(seed)** : Définit la graine de l'aléatoire pour rendre les résultats reproductibles.
- **np.random.uniform(a, b, size)** : Génère des nombres aléatoires selon une distribution uniforme entre **a** et **b**.
- **np.random.rand(d0, d1, ...)** / **np.random.randn(...)** : Distributions uniformes [0,1) ou normales (gaussiennes).

Lecture de Fichiers :

- **np.loadtxt(fname, delimiter, skiprows)** : Charge des données depuis un fichier texte (comme un CSV).
 - **delimiter=','** : Indique le séparateur de colonnes (souvent la virgule).
 - **skiprows=1** : Essentiel pour ignorer la ou les premières lignes (ex: en-têtes de colonnes textuels).

Matplotlib (import matplotlib.pyplot as plt)

Bibliothèque standard pour la création de graphiques et la visualisation de données.

Création et Structure :

- **plt.figure(figsize=(l, h))** : Initialise une figure avec une taille spécifique.
- **plt.subplots(nrows, ncols, figsize)** : Crée à la fois la figure principale et plusieurs sous-graphiques ("Axes") selon une grille.

Types de Graphiques 2D :

- **plt.plot(x, y, color, label, linestyle, marker)** : Trace une courbe reliant les points. Arguments utiles pour l'apparence et la légende.
- **plt.scatter(x, y, c, s, marker)** : Trace un nuage de points sans les relier. **c** pour la couleur, **s** pour la taille.
- **plt.hist(data, bins)** : Trace un histogramme (distribution empirique d'un échantillon).
- **plt.loglog(x, y)** : Échelle logarithmique sur les deux axes (*x* et *y*).
- **plt.semilogx(x, y)** (ou **semilogy**) : Échelle logarithmique sur un seul axe.

- `plt.axvline(x, color, linestyle)` / `plt.axhline(y)` : Trace une ligne d'une extrémité à l'autre du graphique, verticale ou horizontale.
- `plt.contourf(X, Y, Z, cmap)` : Dessine des courbes de niveau remplies (cartes de chaleur 2D). `cmap` définit la palette de couleurs.
- `plt.pcolormesh(X, Y, Z, cmap, shading)` : Grille colorée pseudo-couleur (une alternative plus rapide à `contourf`).

Configuration du Graphique :

- `plt.title(label)`, `plt.xlabel(xlabel)`, `plt.ylabel(ylabel)` : Ajout du titre et des légendes d'axes.
– *Note*: Sur un objet Axe (via `subplots`), on utilise `ax.set_title()`, `ax.set_xlabel()`, etc.
- `plt.grid(True)` : Affiche la grille de repère.
- `plt.legend()` : Génère la légende (fonctionne si `label="..."` a été défini lors du traçage).
- `plt.colorbar()` : Ajoute une échelle de barres de couleurs au bord d'un `contourf` ou `pcolormesh`.
- `plt.annotate(text, xy)` : Ajoute une annotation textuelle pointant vers des coordonnées `xy` spécifiques.
- `plt.tight_layout()` : Ajuste automatiquement l'espace de la fenêtre pour éviter que les éléments ne débordent.
- `plt.show()` : Affiche physiquement la fenêtre graphique.

Graphiques 3D (`mpl_toolkits.mplot3d`) :

- `fig.add_subplot(111, projection='3d')` : Crée un repère tridimensionnel (Objet axe 3D).
- `ax.plot_surface(X, Y, Z, cmap)` : Trace une surface lisse en 3 dimensions à partir de grilles de coordonnées.

Autres Modules Standards

- `time` :
– `time.perf_counter()` : Horloge avec la plus haute précision disponible (idéal pour mesurer précisément le temps d'exécution d'un bloc de code).
- `os` / `sys` :
– `os.chdir(path)` : Change le dossier de travail actuel (Current Working Directory). Utile si les fichiers de données ne sont pas trouvés.

SciPy — Interpolation (`scipy.interpolate`)

Interpolation 1D

- `scipy.interpolate.CubicSpline(x, y, bc_type='not-a-knot')` : Construit un spline cubique par morceaux passant exactement par tous les points `(x, y)`. Produit une courbe lisse avec continuité des dérivées 1ère et 2ème.
– `cs(x_new)` : Évalue le spline en de nouveaux points.
– `cs(x_new, nu=1)` : Dérivée première dy/dx .
– `cs(x_new, nu=2)` : Dérivée seconde d^2y/dx^2 .
– `cs.integrate(a, b)` : Calcule $\int_a^b y(x) dx$ de manière exacte.
– Options `bc_type` : 'not-a-knot' (défaut), 'natural' ($f'' = 0$ aux bords), 'clamped' ($f' = 0$ aux bords), 'periodic' ($f(a) = f(b)$ pour les bords a et b).
- `scipy.interpolate.make_smoothing_spline(x, y, lam=None)` : Spline de lissage pour données bruitées — passe près des points sans suivre le bruit.
– `lam` : paramètre de lissage (grand $\rightarrow$ plus lisse, petit $\rightarrow$ suit de près les données).

Interpolation 2D

- `scipy.interpolate.RegularGridInterpolator((x_grid, y_grid), Z)` : Interpolation sur une grille régulière 2D. $Z[i, j] = f(x_grid[i], y_grid[j])$. Appel : `interp([[x_new, y_new]])`.
- `scipy.interpolate.griddata(points, values, (xi, yi), method='linear')` : Interpolation sur un nuage de points éparses (coordonnées irrégulières).
– `points` : tableau (N, 2) des coordonnées mesurées.
– `values` : tableau (N,) des valeurs mesurées.
– `(xi, yi)` : grille régulière de destination, créée avec `np.meshgrid`.
– `method` : 'linear', 'cubic', ou 'nearest'.

SciPy — Optimisation et Recherche des Racines (`scipy.optimize`)

- **Bissection** : Divise `[a, b]` par deux à chaque étape en conservant la moitié qui contient la racine. Convergence linéaire et garantie de converger si la racine est encadrée.

- **Newton–Raphson** : $x_{n+1} = x_n - f(x_n)/f'(x_n)$. Convergence quadratique, mais nécessite la dérivée analytique **df** et un bon point de départ **x0**.
- **Sécant** : $x_{n+1} = x_n - f(x_n) \cdot \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}$. Convergence superlinéaire, robuste mais nécessite deux points de départ x_1 et x_0 .
- **scipy.optimize.brentq(f, a, b)** : Trouve la racine unique de **f** dans **[a, b]** (algorithme hybride de Brent).
 - Prérequis : **f(a)** et **f(b)** de signes opposés (encadrement valide).
 - Combine bisection (robustesse), méthode de la sécante et interpolation quadratique inverse (rapidité).

SciPy — Systèmes d'équations non linéaires (**scipy.optimize**)

On cherche $\mathbf{x}^* \in \mathbb{R}^n$ tel que $\mathbf{F}(\mathbf{x}^*) = \mathbf{0}$, où $\mathbf{F} : \mathbb{R}^n \rightarrow \mathbb{R}^n$ est une fonction vectorielle non linéaire.

La matrice Jacobienne L'analogue vectoriel de la dérivée : $J_{ij} = \frac{\partial F_i}{\partial x_j}$

C'est une matrice $n \times n$ qui décrit la sensibilité de chaque composante F_i à un changement de chaque variable x_j .

Newton–Raphson vectoriel (implémentation manuelle) À chaque itération, on résout le système linéaire $J(\mathbf{x}_k) \Delta \mathbf{x} = -\mathbf{F}(\mathbf{x}_k)$, puis on met à jour $\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta \mathbf{x}$.

```
x = x0.copy()
for k in range(max_iter):
    Fk = F(x)
    if np.linalg.norm(Fk) < tol:
        break
    dx = np.linalg.solve(J(x), -Fk)
    x = x + dx
```

- Convergence **quadratique** près de la solution : le nombre de décimales correctes double à chaque itération.
- Nécessite un bon point de départ ; peut diverger si on part loin de la solution.

Jacobien numérique (approximation par différences finies) Quand le Jacobien analytique est difficile à écrire, on l'approche avec n évaluations supplémentaires de **F** :

$$\frac{\partial F_i}{\partial x_j} \approx \frac{F_i(\mathbf{x} + \delta \mathbf{e}_j) - F_i(\mathbf{x})}{\delta}$$

```
def jacobien_numerique(F, x, delta=1e-8):
    n = len(x)
    F0 = F(x)
    J = np.zeros((n, n))
    for j in range(n):
        x_pert = x.copy()
        x_pert[j] += delta
        J[:, j] = (F(x_pert) - F0) / delta
    return J
```

Recherche linéaire par rétréci (backtracking line search) Quand le pas de Newton $\Delta \mathbf{x}$ est trop grand, on cherche un facteur $\alpha \in (0, 1]$ qui réduit effectivement la norme résiduelle :

```
alpha = 1.0
norme0 = np.linalg.norm(Fk)
while np.linalg.norm(F(x + alpha*dx)) >= norme0:
    alpha *= 0.5
    if alpha < 1e-14:
        break
x = x + alpha * dx
```

`scipy.optimize.root(F, x0, method='hybr')` Solveur généraliste pour les systèmes non linéaires vectoriels.

`from scipy.optimize import root`

`sol = root(F, x0, method='hybr')`

- `sol.success` : booléen indiquant si le solveur a convergé.
- `sol.fun` : valeur de $\mathbf{F}(\mathbf{x}^*)$ à la solution.
- `sol.x` : vecteur solution $\mathbf{x}^*$.

Méthodes disponibles :

Méthode	Description
'hybr'	Hybride Powell–Broyden — robuste, recommandée par défaut
'lm'	Levenberg–Marquardt — très robuste pour les problèmes sur-déterminés : $\Delta \mathbf{x} = -(J^T J + \lambda I)^{-1} J^T \mathbf{F}$. λ adaptatif : Newton quand on est proche, descente de gradient quand on est loin.
'broyden1'	Quasi-Newton (Broyden) — met à jour une approximation $B_k \approx J(\mathbf{x}_k)$ sans recalcul complet. Convergence superlinéaire, moins coûteux pour les grands systèmes.

SciPy — Intégration numérique (`scipy.integrate`)

Quadrature de Newton-Cotes

- `scipy.integrate.trapezoid(y, x)` : Règle des trapèzes composite. Relie les points par des segments droits. Erreur $\mathcal{O}(h^2)$ — diviser h par 2 divise l'erreur par 4.
 - y est le premier argument, x (optionnel, si pas équiréparti) est le deuxième.
- `scipy.integrate.simpson(y, x)` : Règle de Simpson composite. Approche locale par paraboles. Erreur $\mathcal{O}(h^4)$ — deux ordres meilleurs que les trapèzes pour une même régularité.
 - Nécessite un **nombre impair** de points (nombre pair d'intervalles).
 - y est le premier argument, x (optionnel, si pas équiréparti) est le deuxième.

Quadrature de Gauss–Legendre

- `np.polynomial.legendre.leggauss(n)` : Renvoie $(\mathbf{t}, \mathbf{w})$ — les n nœuds $t_i \in [-1, 1]$ et poids w_i optimaux.
- **Changement de variable** pour intégrer sur $[a, b]$:

$$\int_a^b f(x) dx \approx \frac{b-a}{2} \sum_{i=1}^n w_i f\left(\frac{b-a}{2}t_i + \frac{a+b}{2}\right)$$

- **Propriété clé** : n points intègrent **exactement** tout polynôme de degré $\leq 2n-1$ (contre degré 1 pour les trapèzes à n points, degré 3 pour Simpson).

Intégration adaptative

- `scipy.integrate.quad(f, a, b)` : Quadrature de Gauss-Kronrod adaptative — raffine automatiquement là où la fonction varie rapidement.
 - Renvoie **deux valeurs** : (valeur, erreur_estimee).
 - Limites infinies : `quad(f, 0, np.inf)` ou `quad(f, -np.inf, np.inf)`.
 - Singularités : `quad(f, a, b, points=[x0])` — signale l'emplacement d'une singularité pour aider l'algorithme.
- `scipy.integrate.dblquad(f, a, b, gfun, hfun)` : Intégration numérique 2D (ordre d'appel de la fonction : $\mathbf{f}(\mathbf{y}, \mathbf{x})$).
 - Renvoie (valeur, erreur_estimee) comme `quad`.
 - Cas rectangle simple : `dblquad(f, x_min, x_max, y_min, y_max)`.

Intégrale cumulative

- `scipy.integrate.cumulative_trapezoid(y, x, initial=0)` : Calcule $F(x) = \int_a^x f(t) dt$ en fonction de la borne supérieure variable.
 - `initial=0` : fixe $F(x_0) = 0$ et renvoie un tableau de même taille que y .
 - y est le premier argument, x (optionnel, si pas équiréparti) est le deuxième.

SciPy — Probabilités (`scipy.stats`)

- `scipy.stats.norm.pdf(x, loc=0, scale=1)` : Densité de la loi normale (utile pour l'importance sampling gaussien).
 - `scipy.stats.rayleigh.rvs(scale, size, random_state)` : Échantillonnage selon une loi de Rayleigh.
 - `scipy.stats.rayleigh.pdf(x, scale)` : Densité de Rayleigh (pour calculer les poids d'importance).
-

7. Monte Carlo

Idée générale

- Remplacer une grille déterministe par des tirages aléatoires.
- En 1D sur $[a, b]$:

$$\hat{I}_N = (b - a) \frac{1}{N} \sum_{i=1}^N f(x_i), \quad x_i \sim \mathcal{U}([a, b])$$

- Erreur standard estimée :

$$\text{SE}(\hat{I}_N) \approx (b - a) \frac{\sigma_f}{\sqrt{N}}$$

- Règle pratique : pour diviser l'erreur par 10, il faut environ 100 fois plus d'échantillons.

Cas multi-dimensionnel

- Sur un hyperrectangle de volume V :

$$\hat{I}_N = V \frac{1}{N} \sum_{i=1}^N f(\mathbf{x}_i)$$

- Avantage clé : la convergence reste en $\mathcal{O}(1/\sqrt{N})$, indépendamment de la dimension (pas d'explosion en N^d comme une grille régulière).

Réduction de variance

- **Importance sampling** : échantillonner selon une densité $g(x)$ proche de la forme de f .

$$\hat{I}_N = \frac{1}{N} \sum_{i=1}^N \frac{f(x_i)}{g(x_i)}, \quad x_i \sim g$$

- **Rejection sampling** : tirer dans un domaine englobant puis filtrer avec un masque booléen (`mask = condition`).
- **Échantillonnage stratifié** : découper le domaine en sous-intervalles (strates) et moyenner les contributions de chaque strate.

Motifs de code utiles

- Reproductibilité : `rng = np.random.default_rng(seed=...)`.
 - Erreur standard empirique : `np.std(values, ddof=1) / np.sqrt(N)` (ajuster par le volume/largeur d'intégration).
 - Vérification de convergence : tracer l'erreur en `plt.loglog(N_vals, errors)`.
-

8. Équations Différentielles Ordinaires

Forme générale — problème de valeur initiale (IVP)

$$\frac{dy}{dt} = f(t, y), \quad y(t_0) = y_0$$

Objectif : calculer $y(t)$ pour $t \in [t_0, t_f]$ en avançant pas à pas avec un pas h .

Méthode d'Euler explicite — ordre 1

$$y_{n+1} = y_n + h \cdot f(t_n, y_n)$$

- Erreur locale $\mathcal{O}(h^2)$, erreur globale $\mathcal{O}(h)$.

Méthode du point milieu (RK2) — ordre 2

$$k_1 = f(t_n, y_n), \quad k_2 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right)$$

$$y_{n+1} = y_n + h \cdot k_2$$

- Erreur locale $\mathcal{O}(h^3)$, erreur globale $\mathcal{O}(h^2)$.

Runge-Kutta d'ordre 4 (RK4) — ordre 4

$$k_1 = f(t_n, y_n), \quad k_2 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right)$$

$$k_3 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_2\right), \quad k_4 = f(t_n + h, y_n + hk_3)$$

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

- Erreur locale $\mathcal{O}(h^5)$, erreur globale $\mathcal{O}(h^4)$.

Méthode d'Euler implicite (backward) — ordre 1 :

$$y_{n+1} = y_n + h \cdot f(t_{n+1}, y_{n+1}) \quad (\text{équation implicite})$$

- **A-stable** : stable pour tout h .
- Nécessite de résoudre une équation non linéaire à chaque pas (par ex. Newton-Raphson).
- Erreur globale $\mathcal{O}(h)$ — même ordre qu'Euler explicite, mais sans contrainte de stabilité.

Résumé des ordres de convergence

Méthode	Erreur locale	Erreur globale
Euler	$\mathcal{O}(h^2)$	$\mathcal{O}(h)$
RK2 (point milieu)	$\mathcal{O}(h^3)$	$\mathcal{O}(h^2)$
RK4 classique	$\mathcal{O}(h^5)$	$\mathcal{O}(h^4)$

SciPy — `scipy.integrate.solve_ivp`

Solveur d'EDO avec **pas adaptatif** automatique :

```
from scipy.integrate import solve_ivp
```

```
sol = solve_ivp(f, [t0, tf], [y0], method='RK45', rtol=1e-6, atol=1e-9, dense_output=True)
```

- `f(t, y)` : la condition initiale `[y0]` doit toujours être un **tableau**.
- `method` : 'RK45' (défaut, explicite), 'Radau' ou 'BDF' (implicites).
- `rtol, atol` : tolérances relative et absolue sur l'erreur locale.
- `dense_output=True` : active l'interpolation continue — évaluer ensuite avec `sol.sol(t_new)`.
- `sol.t, sol.y` : tableaux des pas retenus et des valeurs correspondantes.
- `sol.nfev` : nombre total d'évaluations de f .

Systèmes d'EDO — réduction d'ordre

Toutes les méthodes vues (`solve_ivp`, RK4, Euler...) s'appliquent aux systèmes d'EDO avec plusieurs variables, sans modification, à condition que `f` renvoie un vecteur.

Toute EDO d'ordre supérieur peut se convertir à un système d'EDO du premier ordre en rajoutant une variable au premier membre qui est la première dérivée de la première variable dans le premier membre de l'EDO originale.

Schémas d'intégration pour la mécanique

En dynamique des structures, la MEF produit un système semi-discret :

$$\mathbf{M}\ddot{\mathbf{u}} + \mathbf{C}\dot{\mathbf{u}} + \mathbf{K}\mathbf{u} = \mathbf{F}(t)$$

avec $\mathbf{M}$ (masse), $\mathbf{C}$ (amortissement, souvent $\alpha\mathbf{M} + \beta\mathbf{K}$), $\mathbf{K}$ (rigidité).

Méthode θ (forme unifiée)

$$\begin{aligned}\mathbf{u}_{n+1} &= \mathbf{u}_n + h[(1-\theta)\mathbf{v}_n + \theta\mathbf{v}_{n+1}] \\ \mathbf{M}\frac{\mathbf{v}_{n+1} - \mathbf{v}_n}{h} &= -\mathbf{K}[(1-\theta)\mathbf{u}_n + \theta\mathbf{u}_{n+1}]\end{aligned}$$

θ	Schéma	Stabilité
0	Euler explicite	Conditionnelle
1/2	Point au milieu implicite	Inconditionnelle
1	Euler implicite	Inconditionnelle

Pour $\theta \geq 1/2$: **inconditionnellement stable** — le pas de temps n'est limité que par la précision souhaitée.

Méthode de Newmark- β

$$\begin{aligned}\mathbf{u}_{n+1} &= \mathbf{u}_n + h\mathbf{v}_n + h^2[(\tfrac{1}{2} - \beta)\mathbf{a}_n + \beta\mathbf{a}_{n+1}] \\ \mathbf{v}_{n+1} &= \mathbf{v}_n + h[(1-\gamma)\mathbf{a}_n + \gamma\mathbf{a}_{n+1}] \\ \mathbf{M}\mathbf{a}_{n+1} + \mathbf{C}\mathbf{v}_{n+1} + \mathbf{K}\mathbf{u}_{n+1} &= \mathbf{F}_{n+1}\end{aligned}$$

- $\beta = 1/4, \gamma = 1/2$ (accélération moyenne) : **inconditionnellement stable**, ordre 2.
- $\beta = 0, \gamma = 1/2$ (différences centrales) : **explicite**, conditionnellement stable ($h \leq 2/\omega_{\max}$).
- **Attention MEF** : raffiner le maillage augmente $\omega_{\max}$ et force un plus petit pas pour les schémas explicites.

Méthode generalized- α Extension de Newmark avec dissipation numérique contrôlée sur les hautes fréquences. Paramétrisation par le rayon spectral $\rho_\infty \in [0, 1]$:

$$\alpha_m = \frac{2\rho_\infty - 1}{\rho_\infty + 1}, \quad \alpha_f = \frac{\rho_\infty}{\rho_\infty + 1}, \quad \gamma = \tfrac{1}{2} - \alpha_m + \alpha_f, \quad \beta = \tfrac{1}{4}(1 - \alpha_m + \alpha_f)^2$$

- $\rho_\infty = 1$: pas de dissipation (équivalent Newmark $\beta = 1/4$).
- $\rho_\infty = 0$: dissipation maximale des hautes fréquences.

Schéma de Störmer–Verlet (leapfrog/saute-mouton) Simple intégrateur **symplectique** d'ordre 2 :

$$\begin{aligned}\mathbf{u}_{n+1} &= \mathbf{u}_n + h\mathbf{v}_n + \frac{h^2}{2}\mathbf{a}_n \\ \mathbf{a}_{n+1} &= \mathbf{M}^{-1}[\mathbf{F}(t_{n+1}) - \mathbf{K}\mathbf{u}_{n+1}] \\ \mathbf{v}_{n+1} &= \mathbf{v}_n + \frac{h}{2}(\mathbf{a}_n + \mathbf{a}_{n+1})\end{aligned}$$

- Conditionnellement stable ($h < 2/\omega_0$), mais **erreur d'énergie bornée** (pas de dérive séculaire).

Symplecticité Un schéma est **symplectique** s'il préserve le volume dans l'espace des phases (théorème de Liouville).

- Schémas **non symplectiques** (Euler, RK4) : l'énergie dérive sur de longues intégrations.
- Schémas **symplectiques** (Störmer–Verlet) : conservent une énergie modifiée exactement — pas de dérive séculaire.

Schéma	Ordre	Stabilité	Symplectique
--------	-------	-----------	--------------

Récapitulatif des schémas

Schéma	Ordre	Stabilité	Symplectique
Euler explicite	1	Cond.	Non
Euler implicite	1	Incond.	Non
Point milieu implicite ($\theta = 1/2$)	2	Incond.	Oui
Newmark $\beta = 1/4, \gamma = 1/2$	2	Incond.	Non
Störmer–Verlet	2	Cond.	Oui
Generalized- α	2	Incond.	Non